Tutorial 00 — Part 2
Organizing Your Project
The professional Python folder layout

Shuvam Banerji Seal

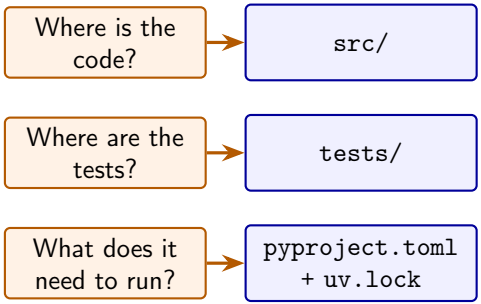
CH4114 — Molecular Simulation

14 August 2026

Repository: `CH4114_Molecular_Simulation_Tutorials`

Why folder structure matters

- One week in, a flat folder becomes unsearchable.
- "Which script made this figure?" must be answerable.
- Structure is how a team (or future-you) communicates.



Three questions a good layout answers instantly

Code → `src/` · Tests → `tests/`
Dependencies → `pyproject.toml` + `uv.lock`

The canonical layout

```
my_project/
+-- README.md
+-- LICENSE
+-- pyproject.toml
+-- uv.lock
+-- .gitignore
+-- .python-version
+-- src/
|   +-- my_project/
|       +-- __init__.py
|       +-- potentials.py
|       +-- analysis.py
+-- tests/
|   +-- test_potentials.py
+-- docs/
+-- scripts/
+-- assets/
+-- notebooks/
```

- **README** — the front page: what, why, how to run.
- **pyproject.toml** — single source of truth for metadata and dependencies.
- **uv.lock** — machine-generated; commit it.
- **.gitignore** — keep `.venv/`, caches, secrets out.
- **src/** — the importable package.
- **tests/** — verification, outside the package.

Why the src-layout?

fragile

Flat layout

`my_project/`

at root

`import` may pick the local folder instead of the installed package

robust

src-layout

`src/my_project/`

`import` always uses the properly installed package

- Prevents the classic bug: tests pass locally, fail elsewhere.
- The layout used by most modern Python projects — and this repo.

What each file promises

File	Contract
`README.md`	30-second explanation: what, why, how to run
`pyproject.toml`	single source of truth: name, version, Python, deps
`uv.lock`	machine-generated; pins every transitive dependency
`.gitignore`	never commit `.venv/`, caches, secrets
`LICENSE`	who may use it, under what terms
`src/<pkg>/__init__.py`	marks the package; often holds version

Rule of thumb

Generated files $\rightarrow$ out of Git. Files needed to *reproduce* the project $\rightarrow$ in Git.

Naming conventions: be consistent

Thing	Convention	Example
Repositories	kebab/snake	`CH4114_Molecular_Simulation_Tutorials`
Python packages	snake	`ch4114`, `mdtraj`
Python files	snake	`lj_potential.py`
Test files	`test_<module>.py`	`test_lj_potential.py`
Branches	kebab	`fix-neighbor-list`
Tutorial folders	`tutorial_<NN>_<DD-MM-YYYY>`	`tutorial_00_14-08-2026`

Commit messages

```
feat:  add Lennard-Jones potential module
fix:  correct prefactor in Ewald summation
docs:  explain thermostat choice in README
test:  cover neighbor-list edge cases
```

A simulation-specific example

```
md_project/
+-- README.md
+-- pyproject.toml
+-- uv.lock
+-- src/md_project/
|   +-- potentials.py
|   +-- integrators.py
|   +-- ensembles.py
|   +-- io.py
+-- tests/
|   +-- test_potentials.py
|   +-- test_integrators.py
+-- scripts/
|   +-- run_benchmark.py
+-- docs/
|   +-- methodology.md
+-- assets/figures/
```

- Code: LJ, Coulomb, Verlet, thermostats.
- Tests verify physics, not just syntax.
- `scripts/` for one-off benchmarks.
- `docs/methodology.md` — write down what you did and why.
- Data and figures are **regenerated**, not committed.

Anti-patterns to avoid

Anti-pattern	Why it hurts
`analysis_final_v2_really.py`	version control exists — use it
Everything in one folder	nobody can find anything
Committing `.venv/`	huge, machine-specific, breaks everyone
No README	the repo is a black box
`pip freeze > requirements.txt`	pins your whole machine, not the project
Editing `main` directly	no review, no record of intent

Check yourself

1. Why does the package live under `src/` instead of the repo root?
2. Which three files answer "what does this project need to run?"?
3. Name two things that must never be committed to Git.
4. What is the tutorial folder naming convention?

Next up

Part 3: uv — Python environments without the pain.